JavaScript - Day 7

BOM | Events & Event Bubbling | Event Delegation

JSON | Error Handling | localStorage

Student Notes — Real-World Projects & Interview Reference

Based on classroom session — covers browser interaction, user events, data exchange, error prevention, and browser storage.

Topic Overview

In this session, we learned how JavaScript interacts with:

- The Browser (BOM)
- User Actions (Events)
- Event Flow (Bubbling & Delegation)
- Data Exchange using JSON
- Runtime Error Handling
- Browser Storage using localStorage

These concepts are heavily used in real-world web applications and form the foundation for modern JavaScript development.

---

1. Browser Object Model (BOM)

How JavaScript interacts with the browser itself

What is BOM?

> *The Browser Object Model (BOM) allows JavaScript to interact with the browser itself — not just the webpage content.*

DOM	BOM
Controls the webpage	Controls the browser
Works with HTML elements	Works with browser features
Uses document object	Uses window object

Why Do We Need BOM?

- How does Netflix know your screen size?
- How does a website know you are using Chrome?
- How can JavaScript refresh a webpage?

All of this is achieved using BOM.

The Window Object

> *Everything in BOM lives inside the window object. It is the global browser object.*

```
window.alert("Hello");

// Same as:
```

```
alert("Hello");
```

BOM Structure

```
                          window
          ________________|_________________
         |          |          |          |          |
    document    location    screen   navigator   history
```

BOM Sub-Objects

location Object

Provides information about the current URL.

```
window.location.href       // Full URL
window.location.hostname   // Domain name
window.location.protocol   // http or https
window.location.reload()   // Refresh the page
```

Output:

```
https://example.com/about
```

screen Object

Provides information about the user's screen.

```
screen.width    // Screen width in pixels
screen.height   // Screen height in pixels
```

Output:

```
1920
```

navigator Object

Provides browser information.

```
navigator.userAgent   // Browser & OS details
navigator.platform    // OS platform
navigator.language    // Browser language
navigator.onLine      // true if internet connected
```

Output:

```
en-US
```

history Object

Stores browser navigation history.

```
history.back();      // Go to previous page
history.forward();   // Go to next page
```

---

2. Events & Event Handlers

Making webpages respond to user actions

What are Events?

> *Events are actions that happen on a webpage — such as clicking, hovering, typing, or submitting a form.*

- Mouse Click
- Hover
- Keyboard Press
- Form Submit
- Page Load

Why Events?

Without events, code runs immediately. With events, code runs only when the user interacts.

```
// Without events – runs immediately
alert("Hello");

// With events – runs on user click
button.addEventListener("click", () => { ... });
```

Types of Events

1. Form Events

Event	Description
change	Value has changed
input	Fires on every keystroke
focus	Input is selected
blur	Input loses focus
submit	Form is submitted

2. Keyboard Events

Event	Description
keydown	Key is pressed down
keyup	Key is released
keypress	Deprecated — avoid using

3. Mouse Events

Event	Description
mousedown	Mouse button pressed
mouseup	Mouse button released
click	Single click
dblclick	Double click
mouseover	Mouse enters element including children
mouseenter	Mouse enters only this element
mousemove	Mouse moves over element

Event Handling Methods

Method 1: Event Properties (onclick)

```
button.onclick = function() {
    alert("Clicked");
}
```

Method 2: addEventListener() — Preferred

```
button.addEventListener("click", function() {
    alert("Clicked");
});
```

Note: *addEventListener() is preferred because it supports multiple event handlers on the same element.*

Example: Random Background Color Generator

```
button.addEventListener("click", function() {
    let r = Math.floor(Math.random() * 255);
```

```
    let g = Math.floor(Math.random() * 255);
    let b = Math.floor(Math.random() * 255);
    document.body.style.backgroundColor = `rgb(${r},${g},${b})`;
});
```

Example: Increase Font Size

```
btn.addEventListener("click", function() {
    size += 10;
    heading.style.fontSize = size + "px";
});
```

---

3. Event Bubbling

How events travel up through the DOM tree

What is Event Bubbling?

> *Event Bubbling is a behavior where an event triggered on a child element automatically propagates upward through its parent elements.*

Example HTML Structure

```
<div id="grandparent">
    <div id="parent">
        <button id="child">Click Me</button>
    </div>
</div>
```

Bubbling Flow

```
User clicks the button

Child fires ↑
Parent fires ↑
Grandparent fires ↑
```

Output:

```
Child Clicked
Parent Clicked
Grandparent Clicked
```

Event Flow Phases

JavaScript events travel through 3 phases:

Phase	Description	Direction
1. Capturing	Event travels down from document to target	Document → Child
2. Target	Event reaches the actual clicked element	At the element
3. Bubbling	Event travels back up to document	Child → Document

4. Event Delegation

Using one listener to handle many child elements

What is Event Delegation?

> *Event Delegation is a technique where a single event listener is placed on a parent element to handle events triggered by its children — using Event Bubbling.*

The Problem — Without Delegation

```
<ul id="list">
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
</ul>
```

❌ Wrong — three separate listeners needed

```
li1.addEventListener("click", ...)
li2.addEventListener("click", ...)
li3.addEventListener("click", ...)
```

The Solution — With Delegation

✅ Correct — one listener on the parent

```
list.addEventListener("click", function(e) {
    if(e.target.tagName === "LI") {
        console.log(e.target.textContent);
    }
});
```

The Event Object (e)

The event object gives information about what was clicked.

Property	Returns
e.target	The element that was actually clicked
e.target.tagName	The tag name (LI, BUTTON, etc.)
e.target.textContent	The visible text of the element

Benefits of Event Delegation

Benefit	Explanation
Better performance	100 items need only 1 listener, not 100
Dynamic elements	New elements automatically handled
Less code	Cleaner and easier to maintain

---

stopPropagation()

> *stopPropagation() stops an event from bubbling further up the DOM tree after the current element.*

Example

```
child.addEventListener("click", function(e) {
    e.stopPropagation();
    console.log("Only child runs");
});
```

Comparison

Normal Bubbling	After stopPropagation()
Child fires	Child fires
Parent fires	✕Stopped here
Grandparent fires	✕Never reached

---

5. JSON (JavaScript Object Notation)

The standard format for client-server data exchange

Why JSON?

> *Client and Server need a common format to exchange data. JSON is the universal standard for this communication.*

```
Client  ⇄  JSON  ⇄  Server
```

Before JSON — XML

```
<book>
    <title>Book Name</title>
    <author>Author Name</author>
</book>
```

Problems with XML:

❌ Too many tags — verbose and repetitive
❌ Heavy file size
❌ Slower to parse

JSON Example

```
{
    "title": "Book Name",
    "author": "Author Name"
}
```

✅ Lightweight
✅ Faster parsing
✅ Human-readable
✅ Matches JavaScript object structure

JSON Data Types

Type	Example
String	"name": "Yathe"
Number	"age": 24
Boolean	"married": false
Object	"address": { "city": "Vizag" }
Array	"skills": ["JS", "React"]
Null	"bike": null

Complete JSON Example

```
{
    "name": "Yathe",
    "age": 24,
    "married": false,
    "skills": ["JS", "React"],
    "bike": null
}
```

JSON Does NOT Support

❌ Functions
❌ Undefined
❌ Date Objects

JSON Methods

JSON.parse()

Converts a JSON String into a JavaScript Object.

```
const obj = JSON.parse(jsonString);
```

JSON.stringify()

Converts a JavaScript Object into a JSON String.

```
const json = JSON.stringify(obj);
```

JSON Data Flow

```
JavaScript Object
      |
      | JSON.stringify()
      ↓
   JSON String  → Sent to Server

   JSON String  ← Received from Server
      |
      | JSON.parse()
      ↓
JavaScript Object
```

---

6. Error Handling

Preventing application crashes during runtime errors

What is Error Handling?

> *Error Handling prevents applications from crashing when unexpected runtime errors occur. The program can detect the error, handle it gracefully, and continue running.*

Without vs With Error Handling

Without Error Handling	With Error Handling
Program starts	Program starts
Error occurs	Error occurs
❌Program stops completely	✅Error is caught and handled
User sees broken page	Program continues running

---

try...catch...finally

```
try {
    // risky code – may fail
}
catch(error) {
    // runs if an error occurs
}
finally {
    // always runs, error or not
}
```

try block

Place any code that might fail inside the try block.

catch block

The catch block runs only if an error occurs inside try.

finally block

The finally block always executes regardless of success or failure. Useful for cleanup operations.

Example: Safe JSON Parsing

```
try {
    let data = JSON.parse(input);
    console.log(data);
```

```
}
catch(error) {
    console.log("Invalid JSON: " + error.message);
}
finally {
    console.log("Finished processing");
}
```

---

throw — Custom Errors

throw creates a custom error with your own message. Used when you want to reject invalid data before an operation.

```
throw new Error("Age must be 18+");
```

Example: Age Validation

```
function validateAge(age) {
    if(age < 18) {
        throw new Error("Age must be 18+");
    }
    console.log("Age accepted");
}

try {
    validateAge(15);
}
catch(error) {
    console.log(error.message);
}
```

Output:

```
Age must be 18+
```

Optional Chaining (?.)

Optional Chaining (?.) safely accesses nested properties. If a property doesn't exist, it returns undefined instead of throwing a TypeError.

Without Optional Chaining

❌ Risky — throws TypeError if address is undefined

```
user.address.city    // Error if address doesn't exist
```

With Optional Chaining

✅ Safe — returns undefined instead of crashing

```
user.address?.city   // Returns undefined safely
```

Using Fallback Values

```
user.address?.city || "City Not Available"
```

If city doesn't exist, returns the fallback string instead.

---

7. localStorage

Storing persistent data inside the browser

What is localStorage?

> *localStorage allows data to be stored inside the browser permanently until manually removed. Data survives page refreshes and browser restarts.*

Features

✅ Data survives page refresh
✅ Data survives browser restart
✅ Key-Value storage
✅ Part of the Web Storage API

> ***Note:*** *localStorage stores only Strings. Objects must be converted first.*

---

localStorage Methods

Method	Purpose	Example
setItem()	Save data	localStorage.setItem("name", "Yathe")
getItem()	Read data	localStorage.getItem("name")
removeItem()	Delete one item	localStorage.removeItem("name")
clear()	Delete all items	localStorage.clear()

Example — Saving and Reading

```
// Save
```

```
localStorage.setItem("username", "Yathe");
localStorage.setItem("phone", "9876543210");

// Read
let name = localStorage.getItem("username");
console.log(name);

// Remove one
localStorage.removeItem("phone");

// Clear all
localStorage.clear();
```

Storing Objects in localStorage

Objects cannot be stored directly — they must be converted to strings first.

Saving an Object

```
let user = { name: "Yathe", age: 24 };

localStorage.setItem(
    "user",
    JSON.stringify(user)
);
```

Retrieving an Object

```
let user = JSON.parse(
    localStorage.getItem("user")
);

console.log(user.name);
```

localStorage + JSON Flow

```
JavaScript Object
      |
      | JSON.stringify()
      ↓
   localStorage (stored as string)
      |
      | JSON.parse()
      ↓
JavaScript Object
```

Storing Multiple Values (Array)

```
// Read existing list (or empty array if none)
let users = JSON.parse(
    localStorage.getItem("users")
) || [];

// Add new user
users.push(newUser);

// Save back
localStorage.setItem(
    "users",
    JSON.stringify(users)
);
```

Real-World Use Cases

Use Case	What to Store
Dark Mode	User's theme preference (light/dark)
Shopping Cart	Cart items so they survive refresh
Form Drafts	Auto-save long forms while typing
React State Persistence	Restore app state after refresh
Login Remember Me	Store logged-in username

Storage Limit

> *localStorage is limited to approximately 5 MB depending on the browser. It is not suitable for large data.*

Security Warning

⚠ **Warning:** Never store passwords, tokens, or sensitive information in localStorage. It is accessible to any JavaScript on the page, making it vulnerable to XSS attacks.

Common Interview Questions

BOM

1. What is BOM?
2. Difference between BOM and DOM?
3. What is the window object?
4. Explain location, navigator, screen, and history objects.

Events

5. What are events?
6. Difference between onclick and addEventListener()?
7. Explain mouse and keyboard events.

Event Bubbling & Delegation

8. What is Event Bubbling?
9. What are the three event phases?
10. What is Event Delegation?
11. Why is Event Delegation useful?
12. What does stopPropagation() do?

JSON

13. What is JSON?
14. Why is JSON preferred over XML?
15. What is JSON.parse()?
16. What is JSON.stringify()?
17. What data types are supported by JSON?

Error Handling

18. What is try...catch?
19. What is the finally block used for?
20. What is throw?
21. What is Optional Chaining?

localStorage

22. What is localStorage?
23. Difference between localStorage and variables?
24. Why use JSON.stringify() with localStorage?
25. Can localStorage store objects directly?
26. Is localStorage secure for passwords?

Practice Exercises

Exercise 1 — BOM Info Display

Display the current URL, browser language, and screen size using BOM.

Exercise 2 — Random Background

Create a button that changes the page background to a random color on each click.

Exercise 3 — Font Size Controls

Create a font-size increase and decrease application with two buttons.

Exercise 4 — Event Bubbling Demo

Create nested divs and demonstrate Event Bubbling. Then use stopPropagation() to control it.

Exercise 5 — Dynamic List with Event Delegation

Create a dynamic list. Use Event Delegation to handle clicks on all items with one listener.

Exercise 6 — JSON Parser

Build a JSON parser that validates user input and shows a friendly error if the JSON is invalid.

Exercise 7 — Age Validator

Create an age validator using throw, try, and catch.

Exercise 8 — localStorage Notes App

Create a Notes App with:

- Add Note
- View all Notes
- Delete a Note
- Notes must persist after page refresh

Mini Assignment — Student Management System

Build a complete Student Management System that uses all concepts from today.

Feature	Concept Used
Display browser information	BOM
Add student through a form	Events

Feature	Concept Used
Handle student actions with one parent listener	Event Delegation
Store student data as JSON strings	JSON
Validate student details before saving	Error Handling
Persist all students after page refresh	localStorage

Key Takeaways

✅ BOM is used to interact with the browser — window, location, screen, navigator, history.
✅ Events make webpages interactive and respond to user actions.
✅ addEventListener() is the preferred event handling method.
✅ Event Bubbling moves events from child to parent automatically.
✅ Event Delegation improves performance and reduces the number of listeners needed.
✅ stopPropagation() stops an event from bubbling further.
✅ JSON is the standard format for client-server data communication.
✅ JSON.parse() converts a JSON string into a JavaScript object.
✅ JSON.stringify() converts a JavaScript object into a JSON string.
✅ Error Handling prevents application crashes and improves user experience.
✅ Optional Chaining (?.) safely accesses nested properties.
✅ localStorage stores persistent data in the browser even after refresh.
✅ Objects must be converted using JSON.stringify() before storing in localStorage.
✅ localStorage should never be used to store passwords or sensitive information.